Department of Computer Applications
II MCA – Java Programming Lab: Hands-on Practice 10
Abstract Class and Interface

I. Abstract Class – Hands-on Problems

Problem 1: Shape Area Calculator

Create an **abstract class Shape** with:

- a **final variable PI**
- an **abstract method calculateArea()**
- a **concrete method displayShape()**

Create two subclasses:

- Circle
- Rectangle

Requirements:

- Circle should calculate the **area using PI**
- Rectangle should calculate the **area using length and breadth**
- Use a **static variable count** in Shape to track how many shape objects are created.

Driver program should:

- Create objects of both classes
- Display the shape type and area
- Display total objects created.

Problem 2: Employee Salary System

Create an **abstract class Employee** with:

- instance variables name and id
- **final variable COMPANY_NAME**
- **abstract method calculateSalary()**
- a **concrete method displayEmployee()**

Create subclasses:

- FullTimeEmployee
- PartTimeEmployee

Requirements:

- Full-time salary = monthly salary
- Part-time salary = hourlyRate × hoursWorked
- Use a **static variable employeeCount** to count employees.

Driver class should:

- Create multiple employees
- Display employee details and salary.

Problem 3: Vehicle Fuel Efficiency

Create an **abstract class Vehicle** with:

- variable model
- **abstract method fuelEfficiency()**
- **final method displayVehicle()**

Create subclasses:

- Car
- Bike

Requirements:

- Each class should return different fuel efficiency values.
- Maintain a **static variable vehicleCount**.

Driver program should:

- Create objects and display vehicle details.

II. Interface – Hands-on Problems

Problem 1: Payment Processing System

Create an **interface Payment** with methods:

- processPayment(double amount)
- printReceipt()

Create classes:

- CreditCardPayment
- UPIPayment

Requirements:

- Use a **static final variable TRANSACTION_FEE** in the interface.
- Each class must implement the methods.

Driver program:

- Accept payment amount
- Process payment using different methods.

Problem 2: Smart Device Control

Create an **interface SmartDevice** with methods:

- turnOn()
- turnOff()

Create classes:

- SmartLight
- SmartFan

Requirements:

- Include a **static method deviceInfo() in the interface**
- Use a **final variable BRAND_NAME**.

Driver program:

- Control multiple devices.

Problem 3: Online Course Evaluation

Create an **interface Course** with methods:

- enrollStudent()
- calculateGrade()

Create classes:

- ProgrammingCourse
- MathCourse

Requirements:

- Include **static final variable PASS_MARK = 50**
- Each class should implement grading logic.

Driver program:

- Enroll students and display grades.

III. Mixed Concepts (Abstract + Interface)

Problem 1: Bank Account System

Create an **abstract class BankAccount** with:

- variable accountNumber
- **abstract method calculateInterest()**
- **final method displayAccount()**

Create an **interface Transaction** with methods:

- deposit()
- withdraw()

Create class:

- SavingsAccount that extends BankAccount and implements Transaction.

Include:

- **static variable bankName**
- Interest calculation.

Driver program:

- Perform deposit, withdraw and interest calculation.

Problem 2: University Staff System

Create an **abstract class Staff** with:

- variables name, department
- abstract method calculatePay()

Create an **interface Attendance** with:

- markAttendance()

Create classes:

- Professor
- LabAssistant

Requirements:

- Include **final variable UNIVERSITY_NAME**
- Implement attendance tracking.

Problem 3: Online Shopping System

Create an **abstract class Product** with:

- variables productName, price
- **abstract method calculateDiscount()**

Create an **interface Shippable** with:

- calculateShippingCost()

Create classes:

- Electronics
- Clothing

Requirements:

- Include **static variable productCount**
- Include **final tax rate constant**.

Driver program:

- Calculate discount and shipping.